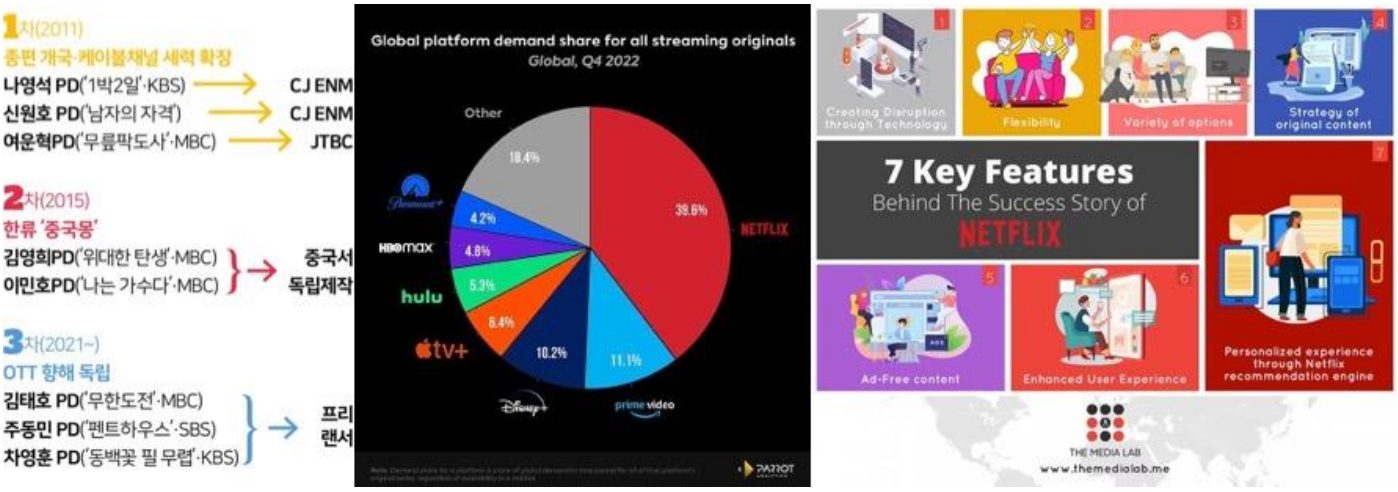
Term Project: Backend Application with PostgreSQL
Team 6

1. 프로젝트의 필요성



OTT의 시대로서 지상파 방송사가 역풍을 맞은 지는 오래 되었다. 유명 PD와 작가들은 10년 전부터 지상파를 떠나고 지금은 OTT 시장으로 몸을 옮기고 있다. 그 OTT 시장에서 항상 부동의 1위인 플랫폼은 넷플릭스이다. 그리고 많은 이들은 이 넷플릭스의 성공 비결 중 하나를 Personalized recommendation engine이라고 한다.

굉장히 복잡한 알고리즘이겠지만 결국은 User의 Rating이 모이고 이를 바탕으로 유사도와 패턴 분석을 통하여 개인화된 추천 컨텐츠를 제공한다. 이 과정에서 Rank by rating / content-based recommendation / collaborative filtering-based recommendation 과 같은 기능은 PostgreSQL 데이터베이스와 상호 작용하는 back-end application 으로 구현하기에 적합하고 현재의 트렌드에도 일치하기에 프로젝트 주제로 정하게 되었다.

2. 프로젝트의 목표

최종목표	**Content recommendation application 구축**
세부목표	**User / Movie 정보 Insert / Delete / Update 구축** **Rating system 구축** **Rank view 구현** **Cold-Start Users 도출** **Content-based recommendation 구현** **Collaborative filtering-based recommendation 구현**

(그림 1. 프로젝트 목표)

Content recommendation application 구축을 위해 6개의 세부 목표를 세워 진행을 하였다. (그림 1).

3. Input data

1) movies 30 rows

mid	title	director	nationality	release_year	profit	summary
1	John Wick: Chapter 4	Chad Stahelski	USA	2023	328,000,000	John Wick faces new enemies in his fight for freedom. High-octane action sequences ensue.
2	No Time to Die	Cary Joji Fukunaga	USA	2021	774,000,000	James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.
3	Black Widow	Cate Shortland	USA	2021	379,000,000	Natasha Romanoff confronts her past as a spy. Explosive stunts and fight scenes dominate.
4	The Suicide Squad	James Gunn	USA	2021	167,000,000	A new team of anti-heroes embark on a dangerous mission. Dark humor and action abound.
5	F9	Justin Lin	USA	2021	726,000,000	Dom Toretto and his crew face new threats from the past. High-speed car chases and action scenes.
6	Shang-Chi and the Legend of the Ten Rings	Destin Daniel Cretton	USA	2021	432,000,000	Shang-Chi discovers his heritage and faces the Ten Rings. Martial arts and action-packed sequences.
7	On Your Wedding Day	Lee Seok-geun	South Korea	2018	11,000,000	A man receives a wedding invitation from his first love, sparking memories of their high school romance and his unresolved feelings.
⋮	⋮	⋮	⋮	⋮	⋮	⋮

(그림 2. Movie data)

Movie에 할당된 id인 mid와 title, director, nationality, release year, profit, summary로 이루어져 있으며 summary는 GPT를 이용하였고 나머지는 수기로 작성하였다. (그림 2).

2) users 15 rows

uid	uname	gender	age
1	Sofia	F	22
2	Liam	M	45
3	Eugene	F	19
4	Hiroshi	M	34
5	Maria	F	27
6	Ahmed	M	28
7	Yana	F	21
⋮	⋮	⋮	⋮

(그림 3. user data)

User에 할당된 id인 uid와 uname, gender, age로 이루어져 있으며 수기로 작성하였다. (그림 3).

3) ratings 230 rows

uid	mid	rating	time
1	1	4.5	2024-06-03
1	2	4.0	2024-06-04
1	4	4.0	2024-05-31
1	5	4.5	2024-04-30
1	6	4.0	2024-05-29
1	7	2.0	2024-05-28
1	8	3.5	2024-04-27
⋮	⋮	⋮	⋮

(그림 4. rating data)

Rating을 한 User의 ID인 uid와 Rating 매겨진 movie의 ID인 mid, rating, time로 이루어져 있으며 수기로 작성하였다. (그림 4).

4. 주요 기능

0) Commands overview

```
Possible commands:
-----------------------------------------------
★ Fetch [ user / movie / rating ]
★ Show [ users / movies / ratings ]
★ Display [ user history / movie ratings ]
-----------------------------------------------
★ Insert [ user / movie / rating ]
★ Update [ user / movie / rating ]
★ Delete [ user / movie / rating ]
-----------------------------------------------
★ Rank [ weekly / monthly ]
-----------------------------------------------
★ Get [ cu / cold-start users ]
★ Rec [ cb / content-based ]
★ Rec [ cf / collaborative filtering ]
-----------------------------------------------
★ Quit

>>>
```

(그림 5. Commands overview)

Application을 구동하면 User가 사용 할 수 있는 기능들이 요약되어 나타남.

User들은 이 commands overview에서 원하는 기능을 찾아서 실행 할 수 있음.

어떠한 기능을 사용해도 하단에 계속 나타나게 하여 Application 이용 시에 commands에 대한 정보를 바로 습득 할 수 있도록 구현 하였음. (그림 5).

1) Conventional Scenarios

해당 혹은 전체 User / movie / Rating의 정보를 요청 경우
새로운 User / movie / Rating을 추가하는 경우
User / movie / Rating을 갱신 혹은 삭제하는 경우
해당 User / movie의 이력을 요청하는 경우

(1) Fetch

```
>>> Fetch user

user id: 1

Fetched user: (1, 'Sofia', 'F', 22)
```

```
>>> Fetch rating

user id: 3
movie id: 4

Fetched rating: (3, 4, 2.0, datetime.date(2024, 5, 25))
```

```
>>> Fetch movie

movie id: 2

Fetched movie: (2, 'No Time to Die', 'Cary Joji Fukunaga', 'USA', 2021, 774000000, 'James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.')
```

(그림 5. Fetch 구동 후의 view)

Fetch User
→ User의 ID를 입력 받아 해당 User의 정보를 나타냄.

Fetch movie
→ Movie의 ID를 입력 받아 해당 Movie의 정보를 나타냄.

Fetch rating
→ User의 ID와 Movie의 ID를 입력 받아 해당 User가 Movie에 매긴 Rating을 나타냄. (그림 5).

```
elif (query == "Fetch user"):          # Keyboard Input이 Fetch user인 경우
    uid = input("user id: ")                   # 조회할 User ID 입력 요청
    try: int(uid)                              # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                         # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                      # 사용자에게 오류 메시지 출력
        continue                                       # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    ret = op.fetch_user(uid)                   # operations.py의 fetch_user 함수를 호출하여 User 정보 조회
    print(f"\nFetched user: {ret}\n")          # 조회된 User 정보 출력

elif (query == "Fetch movie"):         # Keyboard Input이 Fetch movie인 경우
    mid = input("movie id: ")                  # 조회할 Movie ID 입력 요청
    try: int(mid)                              # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                         # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                      # 사용자에게 오류 메시지 출력
        continue                                       # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    ret = op.fetch_movie(mid)                  # operations.py의 fetch_movie 함수를 호출하여 Movie 정보 조회
    print(f"\nFetched movie: {ret}\n")         # 조회된 Movie 정보 출력

elif (query == "Fetch rating"):        # Keyboard Input이 Fetch rating인 경우
    uid = input("user id: ")                   # 조회할 User ID 입력 요청
    try: int(uid)                              # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                         # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                      # 사용자에게 오류 메시지 출력
        continue                                       # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    mid = input("movie id: ")                  # 조회할 Movie ID 입력 요청
    try: int(mid)                              # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                         # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                      # 사용자에게 오류 메시지 출력
        continue                                       # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    ret = op.fetch_rating(uid, mid)            # operations.py의 fetch_rating 함수를 호출하여 Rating 정보 조회
    print(f"\nFetched rating: {ret}\n")        # 조회된 Rating 정보 출력
```

(그림 6. main.py의 Fetch 기능 부분 code. 173행부터 203행 사이에 위치해 있음)

main.py의 구현
→ Keyboard Input을 통해 명령어를 받고 이후에 필요한 ID를 요청함. 입력 값이 정수 혹은 정수 변환이 가능하면 operations.py의 함수를 이용하여 조회함. 만약 입력 값의 정수 변환이 불가능하면 오류 메시지를 출력함. (그림 6).

```python
def fetch_user(uid):                                                    # Keyboard Input을 통해 받은 User ID (uid)에 일치하는 User 정보를 조회하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("SELECT * FROM users WHERE uid = %s", (uid,))                   # SQL 쿼리 실행: users 테이블에서 uid가 일치하는 User의 모든 정보를 조회
    result = cur.fetchone()                                                     # 조회된 행 가져오기
    disconnect(cur, conn)                                                       # PostgreSQL에 연결 종료
    return result                                                               # 조회된 User 정보 리턴

def fetch_movie(mid):                                                   # Keyboard Input을 통해 받은 Movie ID (mid)에 일치하는 Movie 정보를 조회하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("""
        SELECT mid, title, director, nationality, release_year, profit, summary
        FROM movies
        WHERE mid = %s
        """, (mid,))
    result = cur.fetchone()                                                     # SQL 쿼리 실행: movies 테이블에서 mid가 일치하는 Movie의 모든 정보를 조회
    disconnect(cur, conn)                                                       # 조회된 행 가져오기
    return result                                                               # PostgreSQL에 연결 종료
                                                                                # 조회된 Movie 정보 리턴

def fetch_rating(uid, mid):                                             # Keyboard Input을 통해 받은 User ID (uid)와 Movie ID (mid)에 일치하는 Rating 정보를 조회하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("SELECT * FROM ratings WHERE uid = %s AND mid = %s", (uid, mid))  # SQL 쿼리 실행: ratings 테이블에서 uid와 mid가 일치하는 Rating의 모든 정보를 조회
    result = cur.fetchone()                                                     # 조회된 행 가져오기
    disconnect(cur, conn)                                                       # PostgreSQL에 연결 종료
    return result                                                               # 조회된 Rating 정보 리턴
```

(그림 7. operations.py의 Fetch 기능 부분 code. 241행부터 267행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면 PostgreSQL에 연결함. 이후 SQL 쿼리와 결과 처리에 이용 될 커서가 생성되고 SQL 쿼리가 실행됨. SQL 쿼리 로부터 조회된 행을 가져온 뒤 해당 행을 리턴함. (그림 7).

(2) Show

```
>>> Show movies

(1, 'John Wick: Chapter 4', 'Chad Stahelski', 'USA', 2023, 32000000, 'John Wick faces new enemies in his flight for freedom. High-octane action sequences ensue.')
(2, 'No Time to Die', 'Cary Joji Fukunaga', 'USA', 2021, 774000000, 'James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.')
(3, 'Black Widow', 'Cate Shortland', 'USA', 2021, 379000000, 'Natasha Romanoff confronts her past as a spy. Explosive stunts and fight scenes dominate.')
(4, 'The Suicide Squad', 'James Gunn', 'USA', 2021, 167000000, 'A new team of anti-heroes embark on a dangerous mission. Dark humor and action abound.')
(5, 'F9', 'Justin Lin', 'USA', 2021, 726000000, 'Dom Toretto and his crew face new threats from the past. High-speed car chases and action scenes.')
(6, 'Shang-Chi and the Legend of the Ten Rings', 'Destin Daniel Cretton', 'USA', 2021, 432000000, 'Shang-Chi discovers his heritage and faces the Ten Rings. Martial arts and action-packed sequences.')
(7, 'On Your Wedding Day', 'Lee Seok-geun', 'South Korea', 2010, 1100000, 'A man receives a wedding invitation from his first love, sparking memories of their high school romance and his unresolved feelings.')
(8, 'Decision to Leave', 'Park Chan-wook', 'South Korea', 2022, 45000000, 'A detective investigating a man's death in the mountains ends up meeting and developing feelings for the dead man's mysterious wife in the course of his dogged sleuthing. A romance film.')
(9, 'Begin Again', 'John Carney', 'USA', 2013, 6300000, 'A chance encounter between a disgraced music-business executive and a young singer-songwriter new to Manhattan turns into a promising collaboration between the two talents.')
(10, 'La La Land', 'Damien Chazelle', 'USA', 2016, 446300000, 'A jazz musician and an aspiring actress fall in love while navigating their careers in Los Angeles, balancing romance and their dreams in a vibrant, nostalgic musical.')
(11, 'West Side Story', 'Steven Spielberg', 'USA', 2021, 76000000, 'A modern retelling of Romeo and Juliet in 1950s New York. Love and tragedy in a musical format.')
(12, 'A Journal for Jordan', 'Denzel Washington', 'USA', 2021, 6500000, 'A soldier leaves a journal for his son. A love story told through letters.')
(13, 'A Quiet Place Part II', 'John Krasinski', 'USA', 2021, 297000000, 'The Abbott family faces new terrors in silence. Suspense and horror in a post-apocalyptic world.')
(14, 'The Conjuring: The Devil Made Me Do It', 'Michael Chaves', 'USA', 2021, 203000000, 'Paranormal investigators face a new case. Based on true events, it's a chilling and terrifying horror film.')
(15, 'Malignant', 'James Wan', 'USA', 2021, 30000000, 'A woman experiences shocking visions of murders. Horror with a twist.')
(16, 'It', 'Andy Muschietti', 'USA', 2017, 701000000, 'A group of children in Derry, Maine, are terrorized by a shape-shifting entity that takes the form of Pennywise the Clown and face their biggest fears to defeat it.')
(17, 'The Wailing', 'Na Hong-jin', 'South Korea', 2016, 5130000, 'This is a horror film where a mysterious illness starts spreading in a small village, leading a local policeman to enlist the help of a shaman as he becomes increasingly desperate to save his daughter.')
(18, 'Last Night in Soho', 'Edgar wright', 'USA', 2021, 22000000, 'A young woman time-travels to 1960s London. Horror and mystery intertwine.')
(19, 'Encanto', 'Byron Howard', 'USA', 2021, 256000000, 'A magical family in Colombia faces a crisis. A heartwarming and musical animation.')
(20, 'Luca', 'Enrico Casarosa', 'USA', 2021, 49000000, 'Two sea monsters experience an unforgettable summer. Friendship and adventure in Italy crafted into an animation.')
(21, 'Zootopia', 'Byron Howard, Rich Moore', 'USA', 2016, 1024000000, 'An animated film set in a city of anthropomorphic animals, a rookie bunny cop and a cynical con artist fox must work together to uncover a conspiracy.re.')
(22, 'Sing 2', 'Garth Jennings', 'USA', 2021, 407000000, 'Buster Moon and his friends aim for a new stage. Musical comedy animation with a heart.')
(23, 'Madagascar', 'Eric Darnell, Tom McGrath', 'USA', 2005, 556000000, 'An animated film where a group of animals from the Central Park Zoo escape and find themselves stranded on the island of Madagascar, where they must learn to survive in the wild.')
(24, 'Minions', 'Pierre Coffin, Kyle Balda', 'USA', 2015, 1159000000, 'Minions Stuart, Kevin, and Bob are recruited by Scarlet Overkill, a supervillain who, alongside her inventor husband Herb, hatches a plot to take over the world. A thrilling animation.')
(25, 'The Rescue', 'Elizabeth Chai Vasarhelyi', 'USA', 2021, 500000, 'A documentary of the Thai cave rescue. Dramatic and inspiring.')
(26, 'Summer of Soul', 'Questlove', 'USA', 2021, 3700000, 'A look at the 1969 Harlem Cultural Festival. Music and social history intertwine.')
(27, 'Flee', 'Jonas Poher Rasmussen', 'USA', 2021, 1500000, 'A documentary of an Afghan refugee's journey to Denmark. Animated and deeply personal.')
(28, 'Vol', 'Leo Scott', 'USA', 2021, 300000, 'A documentary about actor Val Kilmer. Intimate and revealing documentary.')
(29, 'The Sparks Brothers', 'Edgar Wright', 'USA', 2021, 1600000, 'A look at the band Sparks in a form of documentary. Entertaining and informative.')
(30, 'Procession', 'Robert Greene', 'USA', 2021, 1200000, 'Survivors of sexual abuse create a short documentary. Healing and powerful.')
30 rows in total.
```

```
>>> Show users

(1, 'Sofia', 'F', 22)
(2, 'Liam', 'M', 45)
(3, 'Eugene', 'F', 19)
(4, 'Hiroshi', 'M', 34)
(5, 'Maria', 'F', 27)
(6, 'Ahmed', 'M', 28)
(7, 'Yuna', 'F', 21)
(8, 'Carlos', 'M', 32)
(9, 'Mei', 'F', 24)
(10, 'Ivan', 'M', 50)
(11, 'Rucy', 'F', 33)
(12, 'Luca', 'M', 26)
(13, 'Elly', 'F', 29)
(14, 'Tomas', 'M', 40)
(15, 'Ana', 'F', 20)
 15 rows in total.
```

```
>>> Show ratings

(1, 1, 4.5, datetime.date(2024, 6, 3))
(1, 2, 4.0, datetime.date(2024, 6, 4))
(1, 4, 4.0, datetime.date(2024, 5, 31))
(1, 5, 4.5, datetime.date(2024, 4, 30))
(1, 6, 4.0, datetime.date(2024, 5, 29))
(1, 7, 2.0, datetime.date(2024, 5, 28))
(1, 8, 3.5, datetime.date(2024, 4, 27))
(1, 9, 2.0, datetime.date(2024, 5, 26))
(1, 12, 2.5, datetime.date(2024, 5, 23))
(1, 13, 1.5, datetime.date(2024, 5, 22))
(1, 14, 1.0, datetime.date(2024, 5, 21))
(1, 15, 1.5, datetime.date(2024, 2, 20))
(1, 16, 1.0, datetime.date(2024, 5, 19))
(1, 17, 1.5, datetime.date(2024, 4, 18))
(1, 18, 1.5, datetime.date(2024, 5, 17))
(1, 19, 2.0, datetime.date(2024, 5, 16))
(1, 20, 2.0, datetime.date(2024, 5, 15))
(1, 21, 2.5, datetime.date(2024, 5, 14))
(1, 26, 1.5, datetime.date(2024, 5, 9))
(1, 27, 1.0, datetime.date(2024, 5, 8))
(1, 28, 1.5, datetime.date(2024, 5, 7))
(1, 29, 1.0, datetime.date(2024, 5, 6))
```

(그림 8. Show 구동 후의 view)

Show movies

→ movies 테이블에 저장된 정보와 등록된 Movie의 개수를 나타냄.

Show users

→ users 테이블에 저장된 정보와 등록된 User의 인원을 나타냄.

Show ratings

→ ratings 테이블에 저장된 정보와 등록된 rating의 개수를 나타냄. (그림 8).

```python
elif (query.split()[0] == "Show"):                          # Keyboard Input이 Show로 시작되는 경우
    if (len(query.split()) > 1):                                    # Keyboard Input에 테이블 (users, movies, ratings) 이름이 포함된 경우
        try: ret = op.show(query.split()[1])                                # operations.py의 show 함수 호출하여 해당 테이블 데이터 조회
        except Exception: print(f"{Exception}!\n")                          # 잘못된 테이블 이름 입력과 같은 예외 발생시에 오류 메시지 출력
    if (ret):                                                       # 조회 결과가 있는 경우에는
        for r in ret:                                                       # 조회된 각 행에 대해 for문을 돌려서
            print(r, end='\n')                                                  # 행 데이터 출력
        num = op.count(query.split()[1])                                    # 해당 테이블의 행 개수 계산
        print(f" {num} rows in total.\n")                                   # 계산된 총 행 개수 출력
    else: print("Try again!\n")                                     # 조회 결과가 없는 경우에는 오류 메시지 출력
```

(그림 9. main.py의 Show 기능 부분 code. 173행부터 214행 사이에 위치해 있음)

main.py의 구현

→ Keyboard Input을 통해 받은 명령어의 첫 단어가 Show인 경우에 구동 됨. 명령어의 두번째 단어가 users/movies/ratings 이라면 해당 테이블의 데이터를 조회 함. for문을 이용하여 테이블의 행을 출력하고 해당 테이블의 행 개수도 출력함. users/movies/ratings 이 아니라면 오류 메시지를 출력함. (그림 9).

```python
def show(table_name):                                                   # Keyboard Input을 통해 받은 테이블에 해당하는 모든 데이터를 조회하는 함수
    if (table_name in ["users", "ratings"]):                                # users 혹은 ratings를 테이블 이름으로 받은 경우
        conn = connect_to_db()                                                  # PostgreSQL에 연결
        cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
        str = "SELECT * FROM " + table_name + ";"                               # 구분자 ;를 이용하여 테이블의 모든 데이터를 조회 가능한 SQL 쿼리문 생성
        cur.execute(str)                                                        # SQL 쿼리 실행
        results = cur.fetchall()                                                # 조회된 모든 행 가져오기
        disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
        return results                                                          # 조회된 정보 리턴
    if (table_name == "movies"):                                            # movies를 테이블 이름으로 받은 경우
        conn = connect_to_db()                                                  # PostgreSQL에 연결
        cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
        cur.execute("""
            SELECT mid, title, director, nationality, release_year, profit, summary
            FROM movies
            """)                                                                # SQL 쿼리 실행: movies 테이블에서 embedding 필드를 제외한 모든 데이터 조회
        results = cur.fetchall()                                                # 조회된 모든 행 가져오기
        disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
        return results                                                          # 조회된 정보 리턴
    else: return None                                                       # 테이블 이름이 users/movies/ratings 중에 없는 경우 None 리턴
```

(그림 10. operations.py의 Show 기능 부분 code. 269행부터 288행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면 users/movies/ratings 중에 movies 테이블만 따로 경우를 나누어 진행함. 이후 PostgreSQL에 연결 뒤, SQL 쿼리와 결과 처리에 이용 될 커서를 생성함. movies의 경우 embedding 필드를 제외하고 모든 데이터를 조회하는 SQL문, users/ratings의 경우 구분자를 이용하여 모든 데이터를 조회하는 SQL문 생성. SQL 쿼리 로부터 조회된 모든 행을 가져온 뒤 리턴함. (그림 10).

(3) Display

```
>>> Display user history

user id: 1

User information: (1, 'Sofia', 'F', 22)

Activity history:
(2, 'No Time to Die', 4.0, datetime.date(2024, 6, 4))
(1, 'John Wick: Chapter 4', 4.5, datetime.date(2024, 6, 3))
(4, 'The Suicide Squad', 4.0, datetime.date(2024, 5, 31))
(6, 'Shang-Chi and the Legend of the Ten Rings', 4.0, datetime.date(2024, 5, 29))
(7, 'On Your Wedding Day', 2.0, datetime.date(2024, 5, 28))
(9, 'Begin Again', 2.0, datetime.date(2024, 5, 26))
(12, 'A Journal for Jordan', 2.5, datetime.date(2024, 5, 23))
(13, 'A Quiet Place Part II', 1.5, datetime.date(2024, 5, 22))
(14, 'The Conjuring: The Devil Made Me Do It', 1.0, datetime.date(2024, 5, 21))
(16, 'It', 1.0, datetime.date(2024, 5, 19))
(18, 'Last Night in Soho', 1.5, datetime.date(2024, 5, 17))
(19, 'Encanto', 2.0, datetime.date(2024, 5, 16))
(20, 'Luca', 2.0, datetime.date(2024, 5, 15))
(21, 'Zootopia', 2.5, datetime.date(2024, 5, 14))
(26, 'Summer of Soul', 1.5, datetime.date(2024, 5, 9))
(27, 'Flee', 1.0, datetime.date(2024, 5, 8))
(28, 'Val', 1.5, datetime.date(2024, 5, 7))
(29, 'The Sparks Brothers', 1.0, datetime.date(2024, 5, 6))
(5, 'F9', 4.5, datetime.date(2024, 4, 30))
(8, 'Decision to Leave', 3.5, datetime.date(2024, 4, 27))
(17, 'The Wailing', 1.5, datetime.date(2024, 4, 18))
(15, 'Malignant', 1.5, datetime.date(2024, 2, 20))
```

```
>>> Display movie ratings

movie id: 1

Movie information: (1, 'John Wick: Chapter 4', 'Chad Stahelski', 'USA', 2023, 328000000, '
John Wick faces new enemies in his fight for freedom. High-octane action sequences ensue.'
)

Activity history:
(1, 'Sofia', 4.5, datetime.date(2024, 6, 3))
(11, 'Rucy', 4.5, datetime.date(2024, 6, 3))
(3, 'Eugene', 1.5, datetime.date(2024, 6, 2))
(4, 'Hiroshi', 2.0, datetime.date(2024, 6, 2))
(6, 'Ahmed', 1.0, datetime.date(2024, 6, 2))
(7, 'Yuna', 1.5, datetime.date(2024, 6, 2))
>>> Display movie ratings

movie id: 5

Movie information: (5, 'F9', 'Justin Lin', 'USA', 2021, 726000000, 'Dom Toretto and his
crew face new threats from the past. High-speed car chases and action scenes.')

Activity history:
(2, 'Liam', 5.0, datetime.date(2024, 5, 30))
(7, 'Yuna', 1.5, datetime.date(2024, 5, 25))
(8, 'Carlos', 1.0, datetime.date(2024, 5, 25))
(10, 'Ivan', 1.0, datetime.date(2024, 5, 25))
(4, 'Hiroshi', 2.0, datetime.date(2024, 5, 24))
(3, 'Eugene', 1.5, datetime.date(2024, 5, 24))
(1, 'Sofia', 4.5, datetime.date(2024, 4, 30))
```

(그림 11. Display 구동 후의 view)

Display user history

→ User ID를 입력 받아 해당 User가 조회한 Movie들을 나타냄.

Display movie ratings

→ Movie ID를 입력 받아 해당 Movie를 조회한 User들을 나타냄. (그림 11).

```python
elif (query == "Display user history"):                # Keyboard Input이 Display user history인 경우
    uid = input("user id: ")                                  # 조회할 User ID 입력 요청
    try: int(uid)                                             # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                                        # 정수 변환 실패 시 예외 발생
        print("\nTry again!\n")                                     # 사용자에게 오류 메시지 출력
        continue                                                    # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    ret = op.display_user_history(uid)                        # operations.py의 display_user_history 함수를 호출하여 User 기록 조회
    print(" ")                                                # 빈 줄 출력
    if (ret and ret[0]):                                      # Movie 조회 결과가 있고, User 정보가 존재하는 경우
        print(f"User information: {ret[0]}\n")                      # User 정보 출력
        print(f"Activity history: ")                                # Movie 조회 이력 출력을 구분 하기 위한 헤더 출력
        for r in ret[1]:
            print(r, end='\n')                                              # Movie 조회 이력 출력
    else: print("Cannot find user!")                          # User 정보가 존재하는지 않는 경우 오류 메시지 출력
    print(" ")                                                # 빈 줄 출력

elif (query == "Display movie ratings"):               # Keyboard Input이 Display user history인 경우
    mid = input("movie id: ")                                 # 조회할 Movie ID 입력 요청
    try: int(mid)                                             # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                                        # 정수 변환 실패 시 예외 발생
        print("\nTry again!\n")                                     # 사용자에게 오류 메시지 출력
        continue                                                    # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    print(" ")                                                # 빈 줄 출력
    ret = op.display_movie_ratings(mid)                       # operations.py의 display_movie_ratings 함수를 호출하여 Movie 기록 조회
    if (ret and ret[0]):                                      # User가 Rating한 이력이 있고, Movie 정보가 존재하는 경우
        print(f"Movie information: {ret[0]}\n")                     # Movie 정보 출력
        print(f"Activity history: ")                                # User가 Rating한 이력 출력을 구분 하기 위한 헤더 출력
        for r in ret[1]:
            print(r, end='\n')                                              # User가 Rating한 이력 출력
    else: print("Cannot find movie!")                         # Movie 정보가 존재하는지 않는 경우 오류 메시지 출력
    print(" ")                                                # 빈 줄 출력
```

(그림 12. main.py의 Display 기능 부분 code. 216행부터 245행 사이에 위치해 있음)

main.py의 구현

→ Keyboard Input을 통해 명령어를 받고 이후에 필요한 ID를 요청함. 입력 값이 정수 혹은 정수 변환이 가능하면 operations.py의 함수를 이용하여 조회함. User ID를 입력했다면 해당 User 정보와 Movie 조회 이력을 출력하고, Movie ID를 입력했다면 해당 Movie 정보와 User가 Rating한 이력을 출력함. 만약 입력 값의 정수 변환이 불가능하면 오류 메시지를 출력함. (그림 12).

```python
def display_user_history(uid):                                # Keyboard Input을 통해 받은 User ID (uid)에 일치하는 User 정보와 Movie 조회 이력을 나타내는 함수
    conn = connect_to_db()                                          # PostgreSQL에 연결
    cur = conn.cursor()                                             # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    result = []                                                     # 결과 저장할 리스트
    query1 = """
        SELECT *
        FROM users
        WHERE uid = %s
    """
    cur.execute(query1, (uid,))                                     # User 정보를 조회 가능한 SQL 쿼리문 생성
    result.append(cur.fetchone())                                   # SQL 쿼리 실행
    query2 = """                                                    # 조회된 행 가져오기
        SELECT m.mid, m.title, r.rating, r.time
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        JOIN movies m ON r.mid = m.mid
        WHERE u.uid = %s
        ORDER BY r.time DESC;
    """
    cur.execute(query2, (uid,))                                     # Movie 조회 이력을 탐색 가능한 SQL 쿼리문 생성
    result.append(cur.fetchall())                                   # SQL 쿼리 실행
    disconnect(cur, conn)                                           # 조회된 모든 행 가져오기
    return result                                                   # PostgreSQL에 연결 종료
                                                                    # 조회된 정보 리턴
def display_movie_ratings(mid):                               # Keyboard Input을 통해 받은 Movie ID (mid)에 일치하는 Movie 정보와 User가 Rating한 이력을 나타내는 함수
    conn = connect_to_db()                                          # PostgreSQL에 연결
    cur = conn.cursor()                                             # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    result = []                                                     # 결과 저장할 리스트
    query1 = """
        SELECT mid, title, director, nationality, release_year, profit, summary
        FROM movies
        WHERE mid = %s
    """
    cur.execute(query1, (mid,))                                     # Movie 정보를 조회 가능한 SQL 쿼리문 생성
    result.append(cur.fetchone())                                   # SQL 쿼리 실행
    query2 = """                                                    # 조회된 행 가져오기
        SELECT r.uid, u.uname, r.rating, r.time
        FROM ratings r
        JOIN movies m ON r.mid = m.mid
        JOIN users u ON r.uid = u.uid
        WHERE m.mid = %s
        ORDER BY r.time DESC;
    """
    cur.execute(query2, (mid,))                                     # User가 Rating한 이력을 조회 가능한 SQL 쿼리문 생성
    result.append(cur.fetchall())                                   # SQL 쿼리 실행
    disconnect(cur, conn)                                           # 조회된 모든 행 가져오기
    return result                                                   # PostgreSQL에 연결 종료
                                                                    # 조회된 정보 리턴
```

(그림 13. operations.py의 Display 기능 부분 code. 290행부터 336행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, 이후 PostgreSQL에 연결 뒤, SQL 쿼리와 결과 처리에 이용 될 커서를 생성함. 입력 받은 값이 uid/mid 인지에 따라서 User 정보를 조회와 Movie 조회 이력을 구하는 SQL문, Movie 정보를 조회와 User가 Rating한 이력을 구하는 SQL문을 생성한다. SQL 쿼리 로부터 조회된 행을 가져온 뒤 리턴함. (그림 13).

(4) Insert

```
>>> Insert user

name: SG park
gender [F/M/U]: M
age: 31

Inserted user: (16, 'SG park', 'M', 31)
>>> Fetch user

user id: 16

Fetched user: (16, 'SG park', 'M', 31)
```

Insert user
→ 이름, 성별, 나이를 입력 받아 새로운 User를 생성함. (그림 14).

(그림 14. Insert user 구동 후의 view와 Fetch를 이용한 확인)

```
>>> Insert movie

title: Gone with the Wind
director: Victor Fleming
nationality: United States
release_year: 1939
profit: 390000000
summary: On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney,
and film editor Hal Kern drove out to Riverside, California to preview the film at the F
ox Theatre.

Inserted movie: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 3900
00000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, a
nd film editor Hal Kern drove out to Riverside, California to preview the film at the Fo
x Theatre.')
>>> Fetch movie

movie id: 31

Fetched movie: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 39000
0000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, an
d film editor Hal Kern drove out to Riverside, California to preview the film at the Fox
 Theatre.')
```

Insert movie
→ 이름, 감독, 국가, 연도, 수익, 줄거리를 입력 받아 새로운 Movie를 생성함. (그림 15).

(그림 15. Insert movie 구동 후의 view와 Fetch를 이용한 확인)

```
>>> Insert rating

user id: 16
movie id: 31
rating: 5
time: 2024-06-12

Inserted rating: (16, 31, 5.0, datetime.date(2024, 6, 12))
movie id: 31

Movie information: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 3
90000000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney
, and film editor Hal Kern drove out to Riverside, California to preview the film at the
 Fox Theatre.')

Activity history:
(16, 'SG park', 5.0, datetime.date(2024, 6, 12))
```

Insert rating
→ User ID, Movie ID, 평점, 날짜를 입력 받아 새로운 User를 생성함. (그림 16).

(그림 16. Insert rating 구동 후의 view와 Fetch를 이용한 확인)

```
elif (query == "Insert user"):
    uname = input("name: ")
    gender = input("gender [F/M/U]: ")
    age = input("age: ")
    try:
        ret = op.insert_user(uname, gender, age)
        print(f"\nInserted user: {ret}\n")
    except Exception:
        print("\nWrong values!\n")

elif (query == "Insert movie"):
    title = input("title: ")
    director = input("director: ")
    nationality = input("nationality: ")
    release_year = input("release_year: ")
    profit = input("profit: ")
    summary = input("summary: ")
    try:
        ret = op.insert_movie(title, director, nationality, release_year, profit, summary)
        print(f"\nInserted movie: {ret}\n")
    except Exception:
        print("\nWrong values!\n")

elif (query == "Insert rating"):
    uid = input("user id: ")
    try: int(uid)
    except ValueError:
        print("\nWrong value!\n")
        continue
    mid = input("movie id: ")
    try: int(mid)
    except ValueError:
        print("\nWrong value!\n")
        continue
    rating = input("rating: ")
    time = input("time: ")
    try:
        ret = op.insert_rating(uid, mid, rating, time)
        print(f"\nInserted rating: {ret}\n")
    except Exception:
        print("\nWrong values!\n")
```

```
# Keyboard Input이 Insert user인 경우
        # User 이름 입력 요청
        # User 성별 입력 요청
        # User 나이 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_user 함수 호출하여 User 정보 추가
                # 추가된 User 정보 출력

                # 예외 발생시에 오류 메시지 출력

# Keyboard Input이 Insert movie인 경우
        # movie 제목 입력 요청
        # movie 감독 입력 요청
        # movie 국가 입력 요청
        # movie 연도 입력 요청
        # movie 수익 입력 요청
        # movie 요약 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_movie 함수 호출하여 Movie 정보 추가
                # 추가된 Movie 정보 출력

                # 예외 발생시에 오류 메시지 출력

# Keyboard Input이 Insert rating인 경우
        # rating할 User ID 입력 요청
        # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
        # 정수 변환 실패 시 예외 발생
                # 사용자에게 오류 메시지 출력
                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
        # rating 매겨질 Movie ID 입력 요청
        # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
        # 정수 변환 실패 시 예외 발생
                # 사용자에게 오류 메시지 출력
                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
        # Movie의 rating 입력 요청
        # rating 날짜 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_rating 함수 호출하여 Rating 정보 추가
                # 추가된 Rating 정보 출력

                # 예외 발생시에 오류 메시지 출력
```

(그림 17. main.py의 Insert 기능 부분 code. 45행부터 85행 사이에 위치해 있음)

main.py의 구현

→ Keyboard Input을 통해 명령어를 받고 이후에 필요한 정보를 요청함. 형식에 어긋나지 않는다면 operations.py를 통해 입력한 정보를 새롭게 추가함. 새로운 정보를 출력하거나 만약 형식에 어긋나면 오류 메시지를 출력함. (그림 17).

```python
def insert_user(uname, gender, age):                                    # Keyboard Input을 통해 받은 User 정보를 이용하여 새로운 User를 추가하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    query = """
        INSERT INTO users (uname, gender, age)
        VALUES (%s, %s, %s)
        RETURNING *
    """
    cur.execute(query, (uname, gender, age))                                    # 입력 값을 이용하여 새로운 User를 추가 가능한 SQL 쿼리문 생성
    ret = cur.fetchone()                                                        # SQL 쿼리 실행
    disconnect(cur, conn)                                                       # 추가한 행 가져오기
    return ret                                                                  # PostgreSQL에 연결 종료
                                                                                # 추가된 User 정보 리턴

def insert_movie(title, director, nationality, release_year, profit, summary):  # Keyboard Input을 통해 받은 Movie 정보를 이용하여 새로운 Movie를 추가하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    embedding = embeddings_model.embed_query(summary)                           # OpenAI 임베딩 모델을 사용하여 Movie 요약 정보에 대한 임베딩 벡터 생성

    query = """
        INSERT INTO movies (title, director, nationality, release_year, profit, summary, embedding)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
        RETURNING *
    """
    cur.execute(query, (title, director, nationality, release_year, profit, summary, embedding))  # 입력 값을 이용하여 새로운 Movie를 추가 가능한 SQL 쿼리문 생성
    ret = cur.fetchone()[:-1]                                                   # SQL 쿼리 실행
    disconnect(cur, conn)                                                       # 추가한 행 가져오기
    return ret                                                                  # PostgreSQL에 연결 종료
                                                                                # 추가된 Movie 정보 리턴

def insert_rating(uid, mid, rating, time):                                      # Keyboard Input을 통해 받은 Rating 정보를 이용하여 새로운 Rating를 추가하는 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    query = """
        INSERT INTO ratings (uid, mid, rating, time)
        VALUES (%s, %s, %s, %s)
        RETURNING *
    """
    cur.execute(query, (uid, mid, rating, time))                                # 입력 값을 이용하여 새로운 Rating를 추가 가능한 SQL 쿼리문 생성
    ret = cur.fetchone()                                                        # SQL 쿼리 실행
    disconnect(cur, conn)                                                       # 추가한 행 가져오기
    return ret                                                                  # PostgreSQL에 연결 종료
                                                                                # 추가된 Rating 정보 리턴
```

(그림 18. operations.py의 Display 기능 부분 code. 109행부터 148행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, 이후 PostgreSQL에 연결 뒤, SQL 쿼리와 결과 처리에 이용 될 커서를 생성함. 새로운 User/Movie/Rating 정보를 추가하는 SQL문을 생성하고, SQL 쿼리 로부터 저장된 새로운 정보를 가져온 뒤 리턴함. 이때 Movie의 요약에 대한 정보는 OpenAI 임베딩 모델을 사용하여 임베딩 벡터 생성하는 방식을 이용하였다 (그림 18).

(5) Update

```
>>> Update user

user id: 16
name: PSG
gender [F/M/U]: M
age: 29

Updated user: (16, 'PSG', 'M', 29)
>>> Fetch user

user id: 16

Fetched user: (16, 'PSG', 'M', 29)
```

Update user

→ 이름, 성별, 나이를 입력 받아 User 정보를 갱신함. (그림 19).

(그림 19. Update user 구동 후의 view와 Fetch를 이용한 확인)

```
>>> Update movie

movie id: 31
title: Gone with the Wind
director: VF
nationality: United States
release_year: 1939
profit: 3900
summary: On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney,
and film editor Hal Kern drove out to Riverside, California to preview the film at the F
ox Theatre.

Updated movie: (31, 'Gone with the Wind', 'VF', 'United States', 1939, 3900, 'On Septemb
er 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, and film editor Hal
 Kern drove out to Riverside, California to preview the film at the Fox Theatre.')
>>> Fetch movie

movie id: 31

Fetched movie: (31, 'Gone with the Wind', 'VF', 'United States', 1939, 3900, 'On Septemb
er 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, and film editor Hal
 Kern drove out to Riverside, California to preview the film at the Fox Theatre.')
```

Update movie

→ 이름, 감독, 국가, 연도, 수익, 줄거리를 입력 받아 Movie 정보를 갱신함. (그림 20).

(그림 20. Update movie 구동 후의 view와 Fetch를 이용한 확인)

```
user id: 16
movie id: 31
rating: 4.5
time: 2024-06-12

Updated rating: (16, 31, 4.5, datetime.date(2024, 6, 12))
>>> Fetch rating

user id: 16
movie id: 31

Fetched rating: (16, 31, 4.5, datetime.date(2024, 6, 12))
```

Update rating

→ User ID, Movie ID, 평점, 날짜를 입력 받아 Rating 정보를 갱신함. (그림 21).

(그림 21. Update rating 구동 후의 view와 Fetch를 이용한 확인)

```
elif (query == "Update user"):                                          # Keyboard Input이 Update user인 경우
    uid = input("user id: ")                                                # 갱신할 User ID 입력 요청
    try: int(uid)                                                           # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                                                      # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                                               # 사용자에게 오류 메시지 출력
        continue                                                                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    uname = input("name: ")                                                 # 갱신할 User 이름 입력 요청
    gender = input("gender [F/M/U]: ")                                      # 갱신할 User 성별 입력 요청
    age = input("age: ")                                                    # 갱신할 User 나이 입력 요청
    try:                                                                    # 잘못된 값 입력 방지를 위한 예외처리
        ret = op.update_user(uid=uid, uname=uname, gender=gender, age=age)      # operations.py의 update_user 함수를 호출하여 User 정보 갱신
        print(f"\nUpdated user: {ret}\n")                                        # 갱신된 User 정보 출력
    except Exception:
        print("\nWrong values!\n")                                               # 예외 발생시에 오류 메시지 출력

elif (query == "Update movie"):                                         # Keyboard Input이 Update movie인 경우
    mid = input("movie id: ")                                               # 갱신할 Movie ID 입력 요청
    try: int(mid)                                                           # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                                                      # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                                               # 사용자에게 오류 메시지 출력
        continue                                                                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    title = input("title: ")                                                # 갱신할 Movie의 제목 입력 요청
    director = input("director: ")                                          # 갱신할 Movie의 감독 입력 요청
    nationality = input("nationality: ")                                    # 갱신할 Movie의 국가 입력 요청
    release_year = input("release_year: ")                                  # 갱신할 Movie의 연도 입력 요청
    profit = input("profit: ")                                              # 갱신할 Movie의 수익 입력 요청
    summary = input("summary: ")                                            # 갱신할 Movie의 요약 입력 요청
    try:                                                                    # 잘못된 값 입력 방지를 위한 예외처리
        ret = op.update_movie(mid=mid, title=title, director=director,
                              nationality=nationality, release_year=release_year,
                              profit=profit, summary=summary)                   # operations.py의 update_movie 함수를 호출하여 Movie 정보 갱신
        print(f"\nUpdated movie: {ret}\n")                                       # 갱신된 Movie 정보 출력
    except Exception:
        print("\nWrong values!\n")                                               # 예외 발생시에 오류 메시지 출력

elif (query == "Update rating"):                                        # Keyboard Input이 Update rating인 경우
    uid = input("user id: ")                                                # rating을 갱신할 User ID 입력 요청
    try: int(uid)                                                           # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                                                      # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                                               # 사용자에게 오류 메시지 출력
        continue                                                                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    mid = input("movie id: ")                                               # rating이 갱신 될 Movie ID 입력 요청
    try: int(mid)                                                           # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                                                      # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                                               # 사용자에게 오류 메시지 출력
        continue                                                                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    rating = input("rating: ")
    time = input("time: ")                                                  # rating이 갱신 된 날짜 입력 요청
    try:                                                                    # 잘못된 값 입력 방지를 위한 예외처리
        ret = op.update_rating(uid, mid, rating, time)                          # operations.py의 update_rating 함수를 호출하여 Rating 정보 갱신
```

(그림 22. main.py의 Update 기능 부분 code. 87행부터 139행 사이에 위치해 있음)

main.py의 구현

→ Keyboard Input을 통해 명령어를 받고 갱신에 필요한 정보를 요청함. 형식에 어긋나지 않는다면 operations.py를 통해 입력한 정보를 갱신함. 새로운 정보를 출력하거나 만약 형식에 어긋나면 오류 메시지를 출력함. (그림 22).

```
def update_user(uid, uname=None, gender=None, age=None)                 # Keyboard Input을 통해 받은 User 정보를 이용하여 User 정보를 갱신하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    updates = []                                                            # 갱신할 칼럼과 값을 저장할 리스트
    params = []                                                             # SQL 쿼리에 사용할 파라미터 값 리스트
    if uname:                                                               # 입력받은 정보에 이름이 있다면
        updates.append("uname = %s")                                            # 갱신할 칼럼 목록에 "uname = %s" 추가
        params.append(uname)                                                    # 파라미터 리스트에 uname 추가
    if gender:                                                              # 입력받은 정보에 성별이 있다면
        updates.append("gender = %s")                                           # 갱신할 칼럼 목록에 "gender = %s" 추가
        params.append(gender)                                                   # 파라미터 리스트에 gender 추가
    if age:                                                                 # 입력받은 정보에 나이가 있다면
        updates.append("age = %s")                                              # 갱신할 칼럼 목록에 "age = %s" 추가
        params.append(age)                                                      # 파라미터 리스트에 age 추가
    params.append(uid)                                                      # 파라미터 리스트에 uid 추가
    query = "UPDATE users SET " + ", ".join(updates) + " WHERE uid = %s" + " RETURNING *"   # 입력 값을 이용하여 User 정보를 갱신하는 SQL 쿼리문 생성
    cur.execute(query, tuple(params))                                       # SQL 쿼리 실행
    ret = cur.fetchone()                                                    # 갱신된 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 갱신된 User 정보 리턴

def update_movie(mid, title=None, director=None, nationality=None, release_year=None, profit=None, summary=None):   # Keyboard Input을 통해 받은 Movie 정보를 이용하여 Movie 정보를 갱신하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    updates = []                                                            # 갱신할 칼럼과 값을 저장할 리스트
    params = []                                                             # SQL 쿼리에 사용할 파라미터 값 리스트
    if title:                                                               # 입력받은 정보에 제목이 있다면
        updates.append("title = %s")                                            # 갱신할 칼럼 목록에 "title = %s" 추가
        params.append(title)                                                    # 파라미터 리스트에 title 추가
    if director:                                                            # 입력받은 정보에 감독이 있다면
        updates.append("director = %s")                                         # 갱신할 칼럼 목록에 "director = %s" 추가
        params.append(director)                                                 # 파라미터 리스트에 director 추가
    if nationality:                                                         # 입력받은 정보에 국가가 있다면
        updates.append("nationality = %s")                                      # 갱신할 칼럼 목록에 "nationality = %s" 추가
        params.append(nationality)                                              # 파라미터 리스트에 nationality 추가
    if release_year:                                                        # 입력받은 정보에 연도가 있다면
        updates.append("release_year = %s")                                     # 갱신할 칼럼 목록에 "release_year = %s" 추가
        params.append(release_year)                                             # 파라미터 리스트에 release_year 추가
    if profit:                                                              # 입력받은 정보에 수익이 있다면
        updates.append("profit = %s")                                           # 갱신할 칼럼 목록에 "profit = %s" 추가
        params.append(profit)                                                   # 파라미터 리스트에 profit 추가
    if summary:                                                             # 입력받은 정보에 요약이 있다면
        updates.append("summary = %s")                                          # 갱신할 칼럼 목록에 "summary = %s" 추가
        params.append(summary)                                                  # 파라미터 리스트에 summary 추가
        embedding = embeddings_model.embed_query(summary)                       # OpenAI 임베딩 모델을 사용하여 Movie 요약 정보에 대한 임베딩 벡터 생성
        updates.append("embedding = %s")                                        # 갱신할 칼럼 목록에 "embedding = %s" 추가
        params.append(embedding)                                                # 파라미터 리스트에 embedding 추가
    params.append(mid)                                                      # 파라미터 리스트에 mid 추가
    query = "UPDATE movies SET " + ", ".join(updates) + " WHERE mid = %s" + " RETURNING *"   # 입력 값을 이용하여 Movie 정보를 갱신하는 SQL 쿼리문 생성
    cur.execute(query, tuple(params))                                       # SQL 쿼리 실행
    ret = cur.fetchone()[:-1]                                               # 갱신된 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 갱신된 Movie 정보 리턴

def update_rating(uid, mid, rating=None, time=None):                    # Keyboard Input을 통해 받은 Rating 정보를 이용하여 Rating 정보를 갱신하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    updates = []                                                            # 갱신할 칼럼과 값을 저장할 리스트
    params = []                                                             # SQL 쿼리에 사용할 파라미터 값 리스트
    if rating:                                                              # 입력받은 정보에 평점이 있다면
        updates.append("rating = %s")                                           # 갱신할 칼럼 목록에 "rating = %s" 추가
        params.append(rating)                                                   # 파라미터 리스트에 rating 추가
    if time:                                                                # 입력받은 정보에 날짜가 있다면
        updates.append("time = %s")                                             # 갱신할 칼럼 목록에 "time = %s" 추가
        params.append(time)                                                     # 파라미터 리스트에 time 추가
    params.append(uid)                                                      # 파라미터 리스트에 uid 추가
    params.append(mid)                                                      # 파라미터 리스트에 mid 추가
    query = "UPDATE ratings SET " + ", ".join(updates) + " WHERE uid = %s AND mid = %s" + " RETURNING *"   # 입력 값을 이용하여 Rating 정보를 갱신하는 SQL 쿼리문 생성
    cur.execute(query, tuple(params))                                       # SQL 쿼리 실행
    ret = cur.fetchone()                                                    # 갱신된 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 갱신된 Rating 정보 리턴
```

(그림 23. operations.py의 Update 기능 부분 code. 150행부터 221행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, 이후 PostgreSQL에 연결 뒤, SQL 쿼리와 결과 처리에 이용 될 커서를 생성함. 입력 값을 이용하여 User/Movie/Rating 정보를 갱신을 하는 SQL문을 생성하고, SQL 쿼리 로부터 저장된 새로운 정보를 가져온 뒤 리턴함. 이때 Movie의 요약에 대한 정보는 OpenAI 임베딩 모델을 사용하여 임베딩 벡터 생성하는 방식을 이용하였다 (그림 23).

(6) Delete

```
>>> Delete user

user id: 16
>>> Fetch user

user id: 16

Fetched user: None
```

```
>>> Delete movie

movie id: 31

>>> Fetch movie

movie id: 31

Fetched movie: None
```

```
>>> Delete rating

user id: 16
movie id: 31
>>> Fetch rating

user id: 16
movie id: 31

Fetched rating: None
```

(그림 24. Delete 구동 후의 view와 Fetch를 이용한 확인)

Delete user
→ User ID를 입력 받아 해당 User 정보를 삭제함.

Delete movie
→ Movie ID를 입력 받아 해당 Movie 정보를 삭제함.

Delete rating
→ User ID와 Movie ID를 입력 받아 해당 Rating 정보를 삭제함. (그림 24).

```
elif (query == "Delete user"):          # Keyboard Input이 Delete user인 경우
    uid = input("user id: ")                    # 삭제할 User의 ID 입력 요청
    try: int(uid)                               # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                          # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                       # 사용자에게 오류 메시지 출력
        continue                                        # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    op.delete_user(uid=uid)                     # operations.py의 delete_user 함수를 호출하여 해당 User 정보 삭제
    print(" ")

elif (query == "Delete movie"):         # Keyboard Input이 Delete movie인 경우
    mid = input("movie id: ")                   # 삭제할 Movie의 ID 입력 요청
    try: int(mid)                               # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                          # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                       # 사용자에게 오류 메시지 출력
        continue                                        # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    op.delete_movie(mid=mid)                    # operations.py의 delete_movie 함수를 호출하여 해당 movie 정보 삭제
    print(" ")

elif (query == "Delete rating"):        # Keyboard Input이 Delete rating인 경우
    uid = input("user id: ")                    # 삭제할 rating을 입력한 User의 ID 입력 요청
    try: int(uid)                               # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                          # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                       # 사용자에게 오류 메시지 출력
        continue                                        # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    mid = input("movie id: ")                   # 삭제할 rating의 Movie ID 입력 요청
    try: int(mid)                               # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
    except ValueError:                          # 정수 변환 실패 시 예외 발생
        print("\nWrong value!\n")                       # 사용자에게 오류 메시지 출력
        continue                                        # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    op.delete_rating(uid=uid, mid=mid)          # operations.py의 delete_movie 함수를 호출하여 해당 rating 정보 삭제
    print(" ")
```

(그림 25. main.py의 Delete 기능 부분 code. 140행부터 170행 사이에 위치해 있음)

main.py의 구현
→ Keyboard Input을 통해 명령어를 받고 삭제할 대상의 ID를 요청함. 형식에 어긋나지 않는다면 operations.py를 통해 입력한 ID의 정보를 삭제함. 만약 형식에 어긋나면 오류 메시지를 출력함. (그림 25).

```
def delete_user(uid):                                   # Keyboard Input을 통해 받은 User ID 정보를 이용하여 해당 User 정보를 삭제하는 함수
    conn = connect_to_db()                                      # PostgreSQL에 연결
    cur = conn.cursor()                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("DELETE FROM users WHERE uid = %s", (uid,))     # 입력 값을 이용하여 User 정보를 삭제하는 SQL 쿼리문 생성
    disconnect(cur, conn)                                       # PostgreSQL에 연결 종료

def delete_movie(mid):                                  # Keyboard Input을 통해 받은 Movie ID 정보를 이용하여 해당 Movie 정보를 삭제하는 함수
    conn = connect_to_db()                                      # PostgreSQL에 연결
    cur = conn.cursor()                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("DELETE FROM movies WHERE mid = %s", (mid,))    # 입력 값을 이용하여 Movie 정보를 삭제하는 SQL 쿼리문 생성
    disconnect(cur, conn)                                       # PostgreSQL에 연결 종료

def delete_rating(uid, mid):                            # Keyboard Input을 통해 받은 User ID와 Movie ID 정보를 이용하여 관련 Rating 정보를 삭제하는 함수
    conn = connect_to_db()                                      # PostgreSQL에 연결
    cur = conn.cursor()                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("DELETE FROM ratings WHERE uid = %s AND mid = %s", (uid, mid))  # 입력 값을 이용하여 Rating 정보를 삭제하는 SQL 쿼리문 생성
    disconnect(cur, conn)                                       # PostgreSQL에 연결 종료
```

(그림 26. operations.py의 Update 기능 부분 code. 223행부터 239행 사이에 위치해 있음)

operations.py의 구현
→ main.py를 통해 함수가 호출이 되면, 이후 PostgreSQL에 연결 뒤, SQL 쿼리와 결과 처리에 이용 될 커서를 생성함. 입력 값을 이용하여 User/Movie/Rating를 삭제 하는 SQL문을 이용함 (그림 26).

2) Main Scenario 1 Movie의 Rating에 기반한 Weekly 혹은 Monthly Ranking의 정보를 요청하는 경우

(1) Rank

```
>>> Rank weekly

(5, 'F9', Decimal('5.00'))
(17, 'The Wailing', Decimal('5.00'))
(19, 'Encanto', Decimal('4.83'))
(27, 'Flee', Decimal('4.75'))
(29, 'The Sparks Brothers', Decimal('4.75'))
(21, 'Zootopia', Decimal('4.75'))
(13, 'A Quiet Place Part II', Decimal('4.75'))
(15, 'Malignant', Decimal('4.75'))
(25, 'The Rescue', Decimal('4.67'))
(8, 'Decision to Leave', Decimal('4.50'))
(26, 'Summer of Soul', Decimal('4.50'))
(3, 'Black Widow', Decimal('4.50'))
(23, 'Madagascar', Decimal('4.50'))
(28, 'Val', Decimal('4.25'))
(16, 'It', Decimal('4.25'))
(2, 'No Time to Die', Decimal('4.25'))
(9, 'Begin Again', Decimal('4.25'))
(14, 'The Conjuring: The Devil Made Me Do It', Decimal('4.25'))
(20, 'Luca', Decimal('4.25'))
(10, 'La La Land', Decimal('4.25'))
(22, 'Sing 2', Decimal('4.25'))
(4, 'The Suicide Squad', Decimal('4.25'))
(7, 'On Your Wedding Day', Decimal('4.17'))
(1, 'John Wick: Chapter 4', Decimal('2.86'))
(11, 'West Side Story', None)
(24, 'Minions', None)
(30, 'Procession', None)
(6, 'Shang-Chi and the Legend of the Ten Rings', None)
(18, 'Last Night in Soho', None)
(12, 'A Journal for Jordan', None)
```

```
>>> Rank monthly

(30, 'Procession', Decimal('4.00'))
(11, 'West Side Story', Decimal('3.67'))
(10, 'La La Land', Decimal('3.58'))
(7, 'On Your Wedding Day', Decimal('3.44'))
(8, 'Decision to Leave', Decimal('3.43'))
(12, 'A Journal for Jordan', Decimal('3.43'))
(9, 'Begin Again', Decimal('3.22'))
(29, 'The Sparks Brothers', Decimal('3.13'))
(19, 'Encanto', Decimal('3.00'))
(23, 'Madagascar', Decimal('2.88'))
(1, 'John Wick: Chapter 4', Decimal('2.86'))
(21, 'Zootopia', Decimal('2.85'))
(22, 'Sing 2', Decimal('2.83'))
(15, 'Malignant', Decimal('2.67'))
(20, 'Luca', Decimal('2.60'))
(4, 'The Suicide Squad', Decimal('2.58'))
(17, 'The Wailing', Decimal('2.50'))
(6, 'Shang-Chi and the Legend of the Ten Rings', Decimal('2.43'))
(13, 'A Quiet Place Part II', Decimal('2.38'))
(2, 'No Time to Die', Decimal('2.38'))
(28, 'Val', Decimal('2.31'))
(25, 'The Rescue', Decimal('2.30'))
(24, 'Minions', Decimal('2.25'))
(27, 'Flee', Decimal('2.19'))
(18, 'Last Night in Soho', Decimal('2.06'))
(16, 'It', Decimal('2.00'))
(14, 'The Conjuring: The Devil Made Me Do It', Decimal('2.00'))
(3, 'Black Widow', Decimal('2.00'))
(26, 'Summer of Soul', Decimal('2.00'))
(5, 'F9', Decimal('2.00'))
```

(그림 27. Rank 구동 후의 view)

Rank weekly
→ 한 주간 평점이 높았던 Movie 정보를 정렬함.

Rank monthly
→ 한 달간 평점이 높았던 Movie 정보를 정렬함. (그림 27).

```
elif (query == "Rank weekly"):                    # Keyboard Input이 Rank weekly인 경우
    results = op.show_weekly_ranking()                  # operations.py의 show_weekly_ranking 함수를 호출하여 주간 Movie 순위 조회
    for row in results:
        print(row)                                          # 조회된 정보를 각 행별로 출력

elif (query == "Rank monthly"):                   # Keyboard Input이 Rank monthly인 경우
    results = op.show_monthly_ranking()                 # operations.py의 show_monthly_ranking 함수를 호출하여 월간 Movie 순위 조회
    for row in results:
        print(row)                                          # 조회된 정보를 각 행별로 출력
```

(그림 28. main.py의 Rank 기능 부분 code. 248행부터 256행 사이에 위치해 있음)

main.py의 구현
→Keyboard Input을 통해 명령어를 받고 원하는 Movie 순위를 출력함. (그림 28).

```python
def create_triggers(cur, type):                                                    # Keyboard Input을 통해 받은 주간 랭킹 요청을 수행하기 위한 트리거를 갱신하는 함수

    if(type == "month"):
        create_function_monthly = """
        CREATE OR REPLACE FUNCTION refresh_monthly_ranking() RETURNS TRIGGER AS $$          # create_function_monthly 정의
        BEGIN
            EXECUTE '                                                                       # 월간 순위 뷰를 갱신하는 SQL 쿼리문 생성
                CREATE OR REPLACE VIEW monthly_ranking AS                                   # 월간 순위 뷰 생성 혹은 갱신
                SELECT m.mid, m.title, ROUND(AVG(r.rating::numeric), 2) AS avg_rating       # movie ID, movie title, 평균 평점 정보를 이용
                FROM movies m
                LEFT JOIN ratings r ON m.mid = r.mid AND r.time >= (CURRENT_DATE - 30)      # 테이블에 저장된 평점 날짜를 이용하여 최근 30일 평점만 고려
                GROUP BY m.mid, m.title
                ORDER BY avg_rating DESC NULLS LAST;                                        # 평균 평점은 내림차순 정렬로 나타내고 평점 없는 영화는 마지막으로 배치
            ';
            RETURN NULL;                                                                    # 트리거 함수는 결과를 반환하지 않음
        END;
        $$ LANGUAGE plpgsql;                                                            # PostgreSQL을 이용
        """
        cur.execute(create_function_monthly)                                            # SQL 쿼리 실행

        cur.execute("DROP TRIGGER IF EXISTS refresh_monthly_ranking_trigger_movies ON movies;")   # 기존 트리거 삭제
        create_trigger_movies_monthly = """
            CREATE TRIGGER refresh_monthly_ranking_trigger_movies                           # 트리거 refresh_monthly_ranking_trigger_movies 생성
            AFTER INSERT OR UPDATE OR DELETE ON movies                                      # INSERT, UPDATE, DELETE 발생 시 트리거 실행
            FOR EACH STATEMENT                                                              # 각 행 실행 후에 트리거 실행
            EXECUTE FUNCTION refresh_monthly_ranking();                                     # refresh_monthly_ranking 함수 호출
        """
        cur.execute(create_trigger_movies_monthly)                                      # SQL 쿼리 실행

        cur.execute("DROP TRIGGER IF EXISTS refresh_monthly_ranking_trigger_ratings ON ratings;")   # 기존 트리거 삭제
        create_trigger_ratings_monthly = """
            CREATE TRIGGER refresh_monthly_ranking_trigger_ratings                          # 트리거 refresh_monthly_ranking_trigger_ratings 생성
            AFTER INSERT OR UPDATE OR DELETE ON ratings                                     # INSERT, UPDATE, DELETE 발생 시 트리거 실행
            FOR EACH STATEMENT                                                              # 각 행 실행 후에 트리거 실행
            EXECUTE FUNCTION refresh_monthly_ranking();                                     # refresh_monthly_ranking 함수 호출
        """
        cur.execute(create_trigger_ratings_monthly)                                     # SQL 쿼리 실행

def show_monthly_ranking():                                                             # Keyboard Input을 통해 받은 주간 랭킹 요청을 나타내는 함수
    conn = connect_to_db()                                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("""
        CREATE OR REPLACE VIEW monthly_ranking AS
        SELECT m.mid, m.title, ROUND(AVG(r.rating)::numeric, 2) AS avg_rating
        FROM movies m
        LEFT JOIN ratings r ON m.mid = r.mid AND r.time >= (CURRENT_DATE - 30)
        GROUP BY m.mid, m.title
        ORDER BY avg_rating DESC NULLS LAST;
    """)                                                                                # 트리거 갱신을 위한 함수인 create_function_monthly의 조건과 동일
    create_triggers(cur, "month")                                                       # 트리거 갱신을 위한 함수 호출
    cur.execute("SELECT * FROM monthly_ranking;")                                       # SQL 쿼리 실행
    results = cur.fetchall()                                                            # 조회된 모든 행 가져오기
    disconnect(cur, conn)                                                               # PostgreSQL에 연결 종료
    return results                                                                      # 조회된 정보 리턴
```

(그림 29. operations.py의 Rank monthly 기능 부분 code. Weekly도 유사하다. 335행부터 462행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, 트리거와 함께 Rank에 관련 된 함수가 처리가 된다. 트리거의 작동방식은 우선 처음 순위 정보가 만들어 진 이후에, Insert / Update / Delete 가 수행이 된다면 트리거 실행되어 순위가 갱신된다. 이후 갱신된 정보를 보여주는 함수를 통해 순위가 보이게 된다. (그림 29).

2) Main Scenario 2 Cold-start Users를 위한 Content-based Recommendation을 요청 받은 경우

(1) Cold-Start

```
>>> Get cold-start users

(11, 'Rucy', 'F', 33, 1)
(15, 'Ana', 'F', 20, 1)
(14, 'Tomas', 'M', 40, 1)
(13, 'Elly', 'F', 29, 1)
(12, 'Luca', 'M', 26, 1)
```

```
>>> Insert user

name:  PSG
gender [F/M/U]: M
age: 31

Inserted user: (16, 'PSG', 'M', 31)

>>> Insert rating

user id: 16
movie id: 1
rating: 5
time: 2024-06-12

Inserted rating: (16, 1, 5.0, datetime.date(2024, 6, 12))
```

```
>>> Get cold-start users

(11, 'Rucy', 'F', 33, 1)
(15, 'Ana', 'F', 20, 1)
(14, 'Tomas', 'M', 40, 1)
(13, 'Elly', 'F', 29, 1)
(16, 'PSG', 'M', 31, 1)
(12, 'Luca', 'M', 26, 1)
```

(그림 30. Get cold-start 구동 후의 view와 Insert rating 이후의 cold-start 확인)

Get cold-start

→ Cold-Start User는 보통 활동 이력과 같은 데이터가 분석할 만큼 충분하지 못해서 application이 적합한 결과를 도출하지 못하는 User를 의미함. User들 중에서 이러한 Cold-Start User를 선별하여 나타내는 기능으로서 해당 User의 정보까지 출력함. (그림 30).

```python
def get_cold_start():                                   # Keyboard Input을 통해 받은 Cold-start User 조회 요청을 수행하기 위한 함수
    conn = connect_to_db()                                      # PostgreSQL에 연결
    cur = conn.cursor()                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    query = """                                                 # Cold-start User 조회를 위한 SQL 쿼리문 생성 이때 Rating이 하나도 없는 User는 고려하지 않는다
        SELECT u.uid, u.uname, u.gender, u.age, COUNT(*) as count
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        GROUP BY u.uid, u.uname, u.gender, u.age
        HAVING COUNT(*) < 2                                             # 평점 개수가 2개 미만인 사용자 만을 추출
        ORDER BY count;
        """
    cur.execute(query)                                          # SQL 쿼리 실행
    result = cur.fetchall()                                     # 조회된 모든 행 가져오기
    disconnect(cur, conn)                                       # PostgreSQL에 연결 종료
    return result                                               # 조회된 정보 리턴
```

(그림 31. operations.py의 Get cold-start 기능 부분 code. 466행부터 480행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, User 테이블 중에 평점의 개수가 2개 미만인 사용자 만을 추출하고, 이 User들의 정보를 보인다 (그림 31).

(2) Rec content-based

```
>>> Rec content-based

user id: 16

User information: (16, 'PSG', 'M', 31)

Activity history:
(1, 'John Wick: Chapter 4', 5.0, datetime.date(2024, 6, 12))

Recommended Movies:
(5, 'F9', 'Justin Lin', 'USA', 2021, 726000000, 'Dom Toretto and his crew face new threats from the past. High-speed car chases and action scenes.')
(2, 'No Time to Die', 'Cary Joji Fukunaga', 'USA', 2021, 774000000, 'James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.')
(4, 'The Suicide Squad', 'James Gunn', 'USA', 2021, 167000000, 'A new team of anti-heroes embark on a dangerous mission. Dark humor and action abound.')
```

(그림 32. Rec content-based 구동 후의 view)

(그림 33. John Wick: Chapter 4의 추천 결과)

Rec content-based

→ 앞서 언급한 내용인 Cold-Start User인 경우, User가 평가한 컨텐츠가 매우 적기 때문에 이를 기반으로 컨텐츠를 추천하는 것은 잘못된 추천을 할 위험이 높다. 따라서 이러한 Cold-Start User인 경우에는 해당 User 기반이 아닌 해당 User가 평가한 소수의 컨텐츠의 줄거리를 기반으로 추천을 하였다. 실제 새로 Insert되어 임의의 Cold-Start User가 평가한 John Wick: Chapter 4를 기반으로 한 content-based 추천은 상당히 유사한 장르의 컨텐츠를 추천한다. (그림 33).

User ID를 입력 받아 contene-based 추천을 받는다.

```python
elif (query in ["Rec content-based", "Rec cb"]):        # Keyboard Input이 Rec content-based 혹은 Rec cb인 경우
    uid = input("user id: ")                                    # 추천 받을 User ID 입력 요청
    try: int(uid)                                               # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                                          # 정수 변환 실패 시 예외 발생
        print("\nTry again!\n")                                         # 사용자에게 오류 메시지 출력
        continue                                                        # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    print(" ")
    ret = op.display_user_history(uid)                          # operations.py의 display_user_history 함수를 호출하여 User 기록 조회

    if (ret and ret[0]):                                        # User 정보와 Rating 기록이 존재하는 경우
        print(f"User information: {ret[0]}\n")                          # User 정보 출력
        print(f"Activity history: ")                                    # Rating 기록 출력
        for r in ret[1]:
            print(r, end='\n')
        print("\nRecommended Movies:")                                   # 추천 영화 목록 출력 헤더
        ret = op.recommend_contents_based(uid)                          # operations.py의 recommend_contents_based 함수를 호출하여 contents based 추천 영화 목록 가져오기
        for r in ret: print(r)                                          # contents based 추천 영화 목록 출력
    else: print("Cannot find user!")                            # User 정보 또는 Rating 기록이 존재하지 않는 경우에는 오류 메시지 출력
    print(" ")
```

(그림 34. main.py의 Rec content-based 기능 부분 code. 264행부터 282행 사이에 위치해 있음)

main.py의 구현

→Keyboard Input을 통해 명령어를 받고 입력 받은 User ID를 통해 User 정보를 조회하여 Rating 기록이 있다면, content-based 기반의 추천 영화 목록을 출력함. (그림 34).

```python
def recommend_contents_based(uid):                                  # Keyboard Input을 통해 받은 contents-based 추천 요청을 수행하기 위한 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성

    query1 = """
        SELECT m.embedding
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        JOIN movies m ON r.mid = m.mid
        WHERE u.uid = %s
        ORDER BY r.mid """
    cur.execute(query1, (uid,))                                         # SQL 쿼리 실행: User가 Rating한 영화들의 embedding 벡터 추출
    array1 = np.array(cur.fetchall()).squeeze(axis=1)                   # array1: User가 Rating한 영화들의 embedding 벡터들을 numpy 배열로 저장

    query2 = """
        WITH rated_movies AS (
            SELECT r.mid
            FROM ratings r
            JOIN users u ON r.uid = u.uid
            WHERE u.uid = %s
            ORDER BY r.mid
        )
        SELECT m.embedding
        FROM movies m
        WHERE m.mid NOT IN (SELECT mid FROM rated_movies)
        ORDER BY m.mid
        """
    cur.execute(query2, (uid,))                                         # SQL 쿼리 실행: User가 Rating하지 않은 영화들의 embedding 벡터 추출
    array2 = np.array(cur.fetchall()).squeeze(axis=1)                   # array2: User가 Rating하지 않은 영화들의 embedding 벡터들을 numpy 배열로 저장
    num_unrated = array2.shape[0]                                       # User가 Rating하지 않은 영화들의 개수 저장

    query3 = """
        SELECT r.rating
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        WHERE u.uid = %s
        ORDER BY r.mid """
    cur.execute(query3, (uid,))                                         # SQL 쿼리 실행: User가 Rating한 영화들의 평점을 추출
    array3 = np.array(cur.fetchall()).squeeze(axis=1)                   # User가 Rating한 영화들의 평점을 numpy 배열로 저장
    array3 = np.expand_dims(array3, axis=1)                             # 평점 배열을 2차원으로 변환
    array3 = np.tile(array3, (1, num_unrated))                          # array3: 평점 배열을 User가 평가하지 않은 영화 개수만큼 반복하여 2차원 배열 생성

    k = 3                                                               # 추천할 영화 개수
    array = np.multiply(np.dot(array1, array2.T), array3)               # rate한 영화들과 (array1) rate하지 않은 영화들의 (array2) embedding들을 dot product한 후 rate만큼 가중치를 곱함
    array = np.sum(array, axis=0)                                       # weighted sum하여 최종 array 생성
    index_order = np.argsort(array)[-k:][::-1].tolist()                 # index_order: 유사도 합을 기준으로 내림차순 정렬된 index_order 생성
    index_order = [x + 1 for x in index_order]                          # 인덱스 1부터 시작하도록 조정
    query = f"""
        WITH rated_movies AS (
            SELECT r.mid
            FROM ratings r
            JOIN users u ON r.uid = u.uid
            WHERE u.uid = {uid}
            ORDER BY r.mid
        ),
        unrated_movies AS (
            SELECT m.mid, m.title, m.director, m.nationality, m.release_year, m.profit, m.summary, ROW_NUMBER() OVER (ORDER BY m.mid) AS row_num
            FROM movies m
            WHERE m.mid NOT IN (SELECT mid FROM rated_movies)
            ORDER BY m.mid
        ),
        selected_movies AS (
            SELECT *
            FROM unrated_movies
            WHERE row_num IN {tuple(index_order)}
        )
        SELECT mid, title, director, nationality, release_year, profit, summary
        FROM selected_movies
        ORDER BY CASE row_num
            {''.join([f'WHEN {i} THEN {index_order.index(i)} ' for i in index_order])}
            ELSE row_num
        END; """

    cur.execute(query)                                                  # SQL 쿼리 실행: 앞서 생성한 index_order를 통해 contents-based 추천영화 k개를 추출
    result = cur.fetchall()                                             # 조회된 모든 행 가져오기
    disconnect(cur, conn)                                               # PostgreSQL에 연결 종료
    return result                                                       # 조회된 정보 리턴
```

(그림 35. operations.py의 Rec content-based 기능 부분 code. 482행부터 559행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, User가 Rating한 영화들의 embedding 벡터, User가 Rating하지 않은 영화들의 embedding 벡터, User가 Rating한 영화들의 Rating을 각각 numpy를 이용하여 array1, array2, array3로 저장 한뒤 array1과 array2를 dot product한 후 array3을 활용한 rate만큼 가중치를 곱함. 이를 weighted sum하여 최종 array 생성하고 Cosine Similarity를 기준으로 내림차순 정렬된 index_order를 통해 contents-based 추천 영화를 추출함. (그림 35).

3) Main Scenario 3 Collaborative Filtering Recommendation을 요청 받은 경우

(1) Rec collaborative filtering

```
>>> Insert rating

user id: 16
movie id: 9
rating: 4.5
time: 2024-06-12

Inserted rating: (16, 9, 4.5, datetime.date(2024, 6, 12))
>>> Insert rating

user id: 16
movie id: 10
rating: 5
time: 2024-06-12

Inserted rating: (16, 10, 5.0, datetime.date(2024, 6, 12))
>>> Insert rating

user id: 16
movie id: 7
rating: 4
time: 2024-06-12

Inserted rating: (16, 7, 4.0, datetime.date(2024, 6, 12))
```

```
>>> Rec collaborative filtering

user id: 16

User information: (16, 'PSG', 'M', 31)

Preferred Movies:
(1, 'John Wick: Chapter 4', 5.0, datetime.date(2024, 6, 12))
(9, 'Begin Again', 4.5, datetime.date(2024, 6, 12))
(10, 'La La Land', 5.0, datetime.date(2024, 6, 12))
(7, 'On Your Wedding Day', 4.0, datetime.date(2024, 6, 12))

Recommended Movies:
(8, 'Decision to Leave', 'Park Chan-wook', 'South Korea', 2022, 45000000, "A detective inve
stigating a man's death in the mountains ends up meeting and developing feelings for the de
ad man's mysterious wife in the course of his dogged sleuthing. A romance film.")
(12, 'A Journal for Jordan', 'Denzel Washington', 'USA', 2021, 6500000, 'A soldier leaves a
 journal for his son. A love story told through letters.')
```

(그림 36. Rec collaborative filtering 구동 후의 view)

(그림 37. Insert rating 이후 collaborative filtering을 이용한 추천 결과)

Rec collaborative filtering

→ Cold-Start User가 아닌 경우 User가 평가한 컨텐츠 기반으로 컨텐츠를 추천하는 것은 합당함. 따라서 이러한 C 경우에는 collaborative filtering을 기반으로 추천을 하였다. 앞선 John Wick: Chapter 4을 새로 Insert rating 를 한 User를 계속 이용하였다. John Wick: Chapter 4와는 결이 다른 영화 세 편을 추가로 Insert rating 하였고 선호하는 영화가 4개로 늘어나서 collaborative filtering에 적합한 User 상태가 되게 하였다. (그림 36).collaborative filtering 결과, 이전의 content-based 추천과는 다른 드라마 장르인 A Journal for Jordan과 Decision to Leave가 추천 되었다. 새로 Rating 한 영화를 잘 받아들여 추천을 함을 알 수 있다. (그림 37).

```
elif (query in ["Rec collaborative filtering", "Rec cf"]):          # Keyboard Input이 Rec collaborative filtering 혹은 Rec cf인 경우
    uid = input("user id: ")                                                # 추천 받을 User ID 입력 요청
    try: int(uid)                                                           # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
    except ValueError:                                                      # 정수 변환 실패 시 예외 발생
        print("\nTry again!\n")                                                 # 사용자에게 오류 메시지 출력
        continue                                                            # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
    print(" ")
    ret = op.display_user_history(uid)                                      # operations.py의 display_user_history 함수를 호출하여 User 기록 조회

    if (ret and ret[0]):                                                    # User 정보와 Rating 기록이 존재하는 경우
        print(f"User information: {ret[0]}\n")                                  # User 정보 출력
        print(f"Preferred Movies: ")                                            # User가 선호하는 영화 출력 헤더
        for r in ret[1]:                                                        # Rating 4.0 이상의 User가 선호하는 영화 출력
            if (r[2] >= 4.0): print(r, end='\n')
        print("\nRecommended Movies:")                                           # 추천 영화 목록 출력 헤더
        ret = op.recommend_collaborative_filtering(uid)                         # operations.py의 recommend_collaborative_filtering 함수를 호출하여 collaborative filtering 기반 추천 영화 목록 가져오기
        for r in ret: print(r)                                                  # collaborative filtering 기반 추천 영화 목록 출력
    else: print("Cannot find user!")                                      # User 정보 또는 Rating 기록이 존재하지 않는 경우에는 오류 메시지 출력
    print(" ")
```

(그림 38. main.py의 Rec collaborative filtering 기능 부분 code. 284행부터 302행 사이에 위치해 있음)

main.py의 구현

→ Keyboard Input을 통해 명령어를 받고 입력 받은 User ID를 통해 User 정보를 조회하여 Rating 기록이 있다면, collaborative filtering 기반의 추천 영화 목록을 출력함. (그림 38).

```
def recommend_collaborative_filtering(uid):                                 # Keyboard Input을 통해 받은 collaborative filtering 추천 요청을 수행하기 위한 함수
    conn = connect_to_db()                                                      # PostgreSQL에 연결
    cur = conn.cursor()                                                         # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성

    target_index = int(uid) - 1                                                 # 추천 대상 User의 인덱스 계산

    cur.execute("SELECT COUNT(DISTINCT uid) FROM users")                        # SQL 쿼리 실행: User 수 계산
    num_users = cur.fetchone()[0]
    cur.execute("SELECT COUNT(DISTINCT mid) FROM movies")                       # SQL 쿼리 실행: Movie 수 계산
    num_movies = cur.fetchone()[0]
    rating_matrix = np.zeros((num_users, num_movies))                           # Rating matrix 생성
    cur.execute("SELECT uid, mid, rating FROM ratings")                         # SQL 쿼리 실행: 모든 Rating 데이터 가져오기
    ratings = cur.fetchall()
    for uid, mid, rating in ratings: rating_matrix[uid-1][mid-1] = rating       # Rating matrix에 Rating 데이터 매칭시키기

    similarity_matrix = cosine_similarity(rating_matrix)                        # Rating matrix의 cosine_similarity를 도출하여 similarity_matrix 생성
    user_similarities = similarity_matrix[target_index]                         # 해당 User와 다른 User들의 similarity를 similarity_matrix를 이용하여 구하기
    most_similar_user_index = np.argsort(user_similarities)[-2]                 # 해당 User와 가장 유사한 User의 인덱스 추출
    target_user_ratings = rating_matrix[target_index]                           # 해당 User의 Rating 정보
    similar_user_ratings = rating_matrix[most_similar_user_index]               # 가장 유사한 User의 Rating 정보

    high_rated_movies = np.where(similar_user_ratings >= 4.5)[0]                # 가장 유사한 User가 4.5 이상으로 Rating한 Movie의 인덱스
    unwatched_movies = np.where(target_user_ratings == 0)[0]                    # 해당 User가 아직 보지 않은 영화 인덱스
    recommended_movies = np.intersect1d(high_rated_movies, unwatched_movies)    # 교집합 추출

    if len(recommended_movies) > 0:                                             # 추천 Movie가 있을 경우
        recommended_movies_list = recommended_movies.tolist()                       # numpy array를 리스트로 변환
        recommended_movies_list = [int(mid) + 1 for mid in recommended_movies_list]     # 각 요소를 int로 변환하여 mid 생성
        format_strings = ','.join(['%s'] * len(recommended_movies_list))
        query = f"SELECT mid, title, director, nationality, release_year, profit, summary FROM movies WHERE mid IN ({format_strings})"
        cur.execute(query, tuple(recommended_movies_list))                          # SQL 쿼리 실행: 추천 영화 정보 조회
        movies = cur.fetchall()
    else:
        movies = []                                                                 # 추천 영화가 없을 경우 빈 리스트 반환

    disconnect(cur, conn)                                                       # PostgreSQL에 연결 종료
    return movies                                                               # 조회된 정보 리턴
```

(그림 39. operations.py의 Rec collaborative filtering 기능 부분 code. 563행부터 599행 사이에 위치해 있음)

operations.py의 구현

→ main.py를 통해 함수가 호출이 되면, User 수와 Movie 수를 고려한 Rating Matrix를 생성시켜 Rating 데이터와 매칭 시킨다. Rating Matrix의 cosine similarity를 계산하여 만들어진 similarity matrix를 이용하여 해당 User와 다른 User 간의 similarity를 도출한다. 해당 User와 가장 유사한 User가 선호하는 영화 중 해당 User가 Rating하지 않은 영화를 추출하여 collaborative filtering 기반의 추천이 이루어진다 (그림 39).